Team Name : Soulspinning

Team Leader Name : Rangoju Gayatri Sai

Problem Statement : Air-Gapped Predictive Copilot for Secure MPLS Operations

Team Members

Team Leader:

Name:Rangoju Gayatri Sai
College:Mahindra University

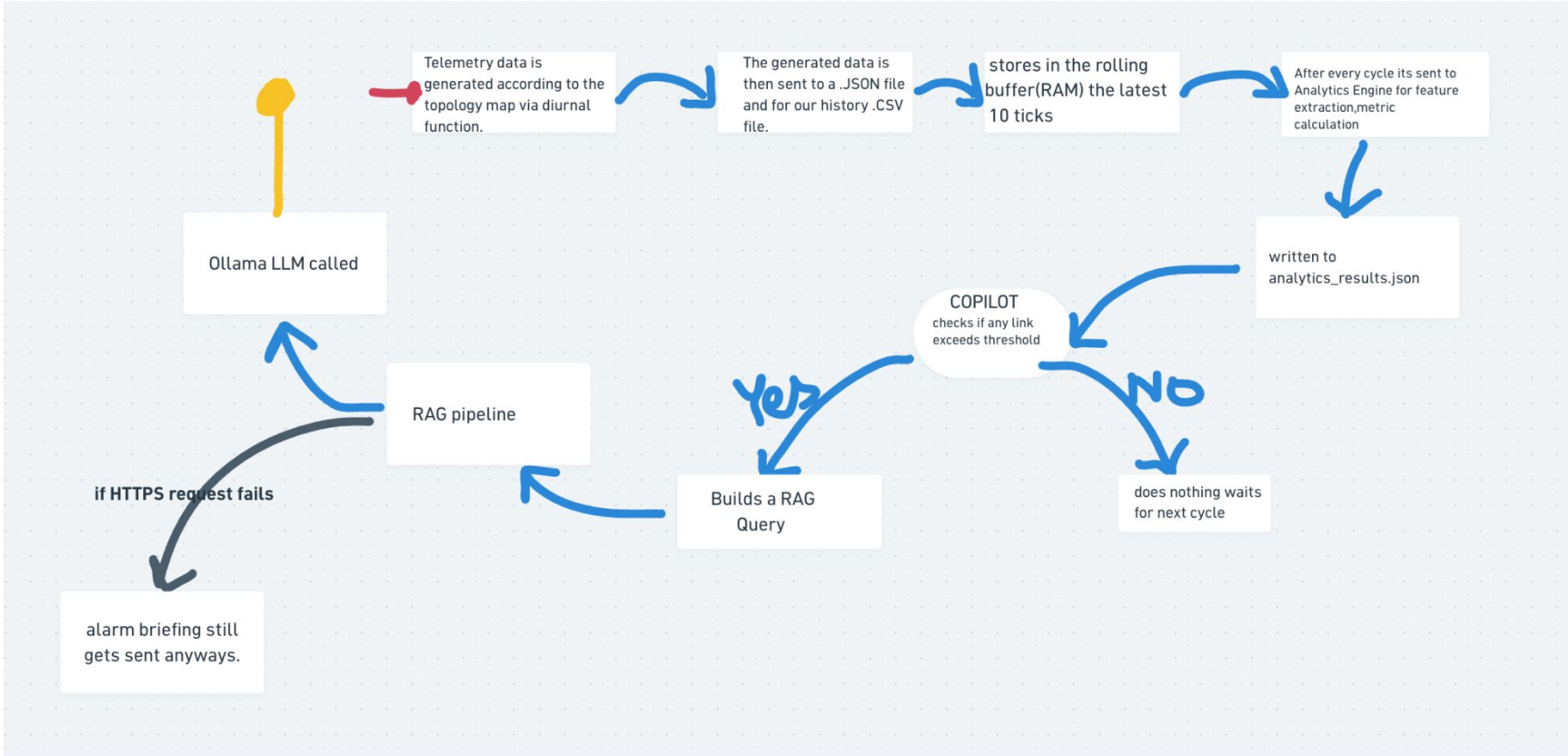
Team Member-1:

Name:Abhinav Reddy Pakanati
College:Mahindra University

Team Member-2:

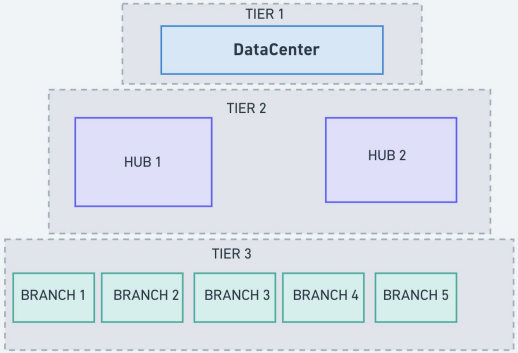
Name:Ritvik Gottimukkala
College:Mahindra University

This is the pileline/the whole workflow of our proposed idea:



Topology map- for the stimulation phase

Network Architecture



Layer 1: The Core Backbone (mpls_core)

- DC1 ⟷ HUB1
  - Bandwidth: 1000 Mbps
  - Latency: 8 ms
- DC1 ⟷ HUB2
  - Bandwidth: 1000 Mbps
  - Latency: 12 ms
- HUB1 ⟷ HUB2
  - Bandwidth: 500 Mbps
  - Latency: 10 ms

Layer 2: The Regional Distribution (mpls_access)

- BRANCH1
  - Bandwidth: 100 Mbps
  - Latency: 15 ms
- BRANCH2
  - Bandwidth: 100 Mbps
  - Latency: 22 ms
- BRANCH3
  - Bandwidth: 100 Mbps
  - Latency: 18 ms
- BRANCH4
  - Bandwidth: 100 Mbps
  - Latency: 14 ms
- BRANCH5
  - Bandwidth: 100 Mbps
  - Latency: 20 ms

Layer 3: The SD-WAN Internet Backups (sdwan_overlay)

- BRANCH1 ⟷ DC1
  - Bandwidth: 50 Mbps
  - Latency: 25 ms
- BRANCH4 ⟷ DC1
  - Bandwidth: 50 Mbps
  - Latency: 28 ms
- BRANCH4 ⟷ HUB1
  - Bandwidth: 50 Mbps
  - Latency: 30 ms
- BRANCH2 ⟷ HUB2
  - Bandwidth: 50 Mbps
  - Latency: 35 ms

Detailed explanation of the flow:

The Telemetry Simulator (Data Generation) :

- the network stimulator wrt to the Topology map every 5 seconds (1 tick), it generates metrics for **all 12 links, 8 BGP lanes, and 4 overlay tunnels** -
- via the diurnal function(this basically uses the sine function to setup the data as it replicates the usual working hours of the day)
- To stimulate the actual network traffic, it uses a random seed to inject faults in between
- Labels /gives tags to the dataset via 1/0 if it falls under [whether a fault is active, if a precursor anomaly is building up]
- Introduces network chaos and noise using a **random seed** to inject unexpected link faults.

The Analytics Engine:

Rolling buffer gets updated and pushed with the following data.[feeds new data ticks]

Once the buffer reaches **10 ticks of history**, the scoring engine runs feature extractions to calculate an overall risk score.

This is now written to the analytics_results.json file

The Copilot:

This file is handed off to the copilot. The copilot just looks over this. Doesn't touch the raw buffer. This polls this file every 3secs irrespective of the time of the arrival of the incoming data.

If any link crosses the safety Risk Threshold, the Copilot automatically translates the numerical alert into a **RAG Query**.

RAG:

The RAG searches over the troubleshooting runbooks and the docs using BM25+TF-IDF via RRF and hands over the results to the Llama 3.1: 8B model.

outputs a plain, natural language explanation [of the plausible upcoming fault] for the NOC.

Why this Architecture?Why did we even take those choices to implement via these elements/components? The Unique Value Proposition[USP] of Our Solution:

***Our system uses Python's native threading library** to implement an asynchronous,multi-threaded pipeline. This multi-threaded execution model ensures that heavy computations do not interleave, **interdependence on each of the individual tasks between the threads, provides synchronization.**

The operational tasks are divided among 3 independent threads mainly:

1) Thread 1: Telemetry Simulation Engine **:**

Role: Runs a continuous background loop(configured to emit every 5 secs) to stimulate network telemetry data, network metrics, for the network topology.(including interface utilization, latency spikes, jitter, packet loss, and BGP routing state anomalies, appending them to files models them dynamically).

2) Thread 2: Streaming Analytics Engine (Processing & Feature Extraction Layer)

Manages an in-memory, fixed-size deque rolling buffer (window size of 10 ticks) per network link inside the RAM workspace. It executes thread safe feature extraction and performs statistical trend math via functions directly from the volatile memory to predict danger before they impact the network.

Role: Takes that streaming data, handles the rolling 10-tick rolling buffer and performs the statistical trend maths directly from the Rolling buffer in the RAM and performs smart calculations, feature extractions.

3) Thread 3: Autonomous Intelligent Copilot

Role: Monitors the analytics results data engine, when the threshold breaks it initializes a Hybrid RAG QUERY.

*Using the laptop's/systems RAM(Random Access Memory)[Volatile memory]

- to maintain a highly volatile, lightweight moving window it stores the most recent 10 ticks.[1tick=5secs](which is equivalent to 50 secs operational window).

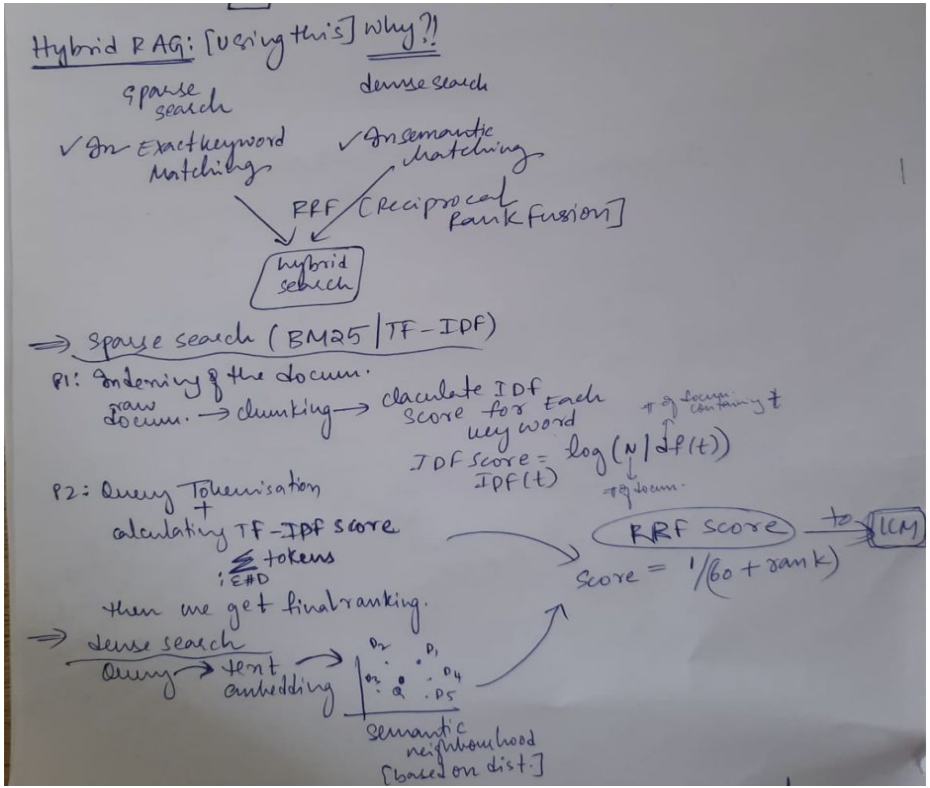
-By calculating analytical features directly within these particular memory arrays, the analytics engine completely avoids the overhead of reading from huge gigabytes of historical CSV or JSON logs during active cycles, also eliminates the need for storage, the time taken for fault detection remains constant regardless of large the topology, the database is.

-This was CPU clock cycles utilization is reduced drastically which allows to process the mathematical calculation upon risk trends even quickly. Which basically tends to free up hardware resources for others processes.

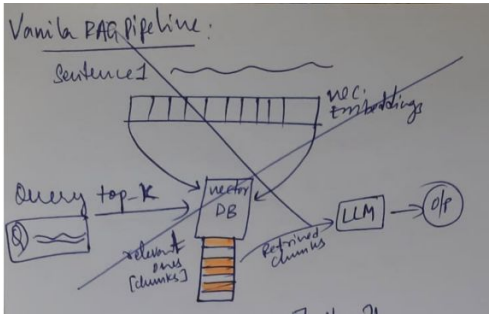
***we use “deque”(for the rolling buffer) specifically instead of a list as**

As this acts as a true circular queue in hardware memory—the moment Tick 11 arrives, Tick 1 disappears from RAM automatically. This guarantees a O(1) [constant time insertion and deletion complexity].

*We will be using Hybrid RAG instead of other categories[such as Vanila RAG, Graph RAG, Agentic RAG]



But why?



Using a hybrid approach combines keyword search (to catch exact technical terms) and semantic search (to understand the underlying meaning).

*Race condition could false trigger a system so

Race Condition Mitigated via the use of **Mutex locks**:

- To prevent critical race conditions we implement **Mutual Exclusion (Mutex) locks** over the volatile in-memory rolling buffers [present in RAM].

-To prevent data corruption, overwriting of data -As the Telemetry Simulator (Thread 1) and the Analytics Engine (Thread 2) access the exact same memory workspace simultaneously.

***We will be using Llama 3.1: 8B (via Ollama)**

For Local, 100% Air-Gapped Compliance, with zero external API dependencies. (satisfies **ISRO's** space-agency security requirements)

Provides a 128K token context window which helps in parsing our multi document RAG data.

EXCITED TO GET THIS OFFLINE ALIVE!


इसरो isro
H2S HACK2SKILL
BHARATIYA ANTARIKSH HACKATHON
2026
THANK YOU